Sentence Style Detection for Author Switches

Author Switch Detection

Tshephang Phuti-A-Nkutu Matlala

PAN 2026 Multi-Author Writing Style Analysis

Outline

1. Representation & features
2. Models overview
3. Results and interpretation
4. What I am still working on
5. Upcoming: estimating the number of authors

Task and data

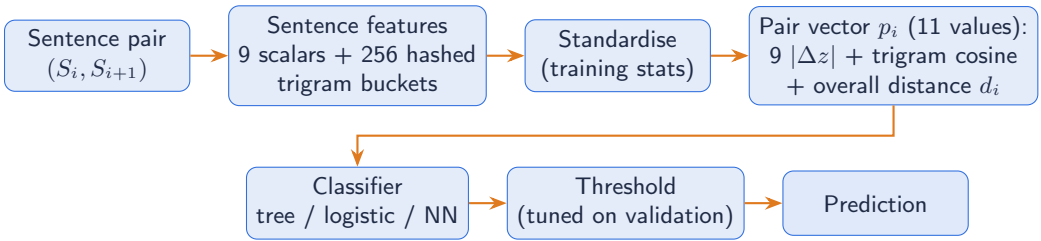
Task: for every pair of adjacent sentences (S_i, S_{i+1}), decide whether the author changes.

- PAN 2026 data
- Easy: topic changes with author. Medium/hard: topic is controlled
- 2,439,074 training pairs, about 14.3% of them are switches

	Pairs	**Switch rate**
Easy	549,804	30.5%
Medium	1,299,324	4.4%
Hard	589,946	20.9%

Switch rates differ sharply by difficulty (easy, medium and hard).

Pipeline: from raw text to a prediction



- Written in Java with *no external ML library*.

Nine style features used during training

Idx	Feature	What it captures
0	Word length ($\log(1 + \#\text{words})$)	Fluency and sentence length
1	Character length	Same, in characters
2	Frequent word ratio (top 10k English)	Habit in the use of function words
3	Punctuation density	Structural and regular writing habits
4	Digit ratio	Formatting habit (digit share)
5	Uppercase ratio	Capitalisation habit
6	Average word length	Vocabulary register
7	Mean TF-IDF	Vocabulary rarity
8	Emotion score (NRC lexicon)	Affective tone

Scalar features are z-scored with training statistics.

Hashed character trigrams and the pair vector

Hashing trick

- Each trigram $\rightarrow$ one of $B = 256$ buckets
- Advantage: fixed length, no corpus required
- Cost: there are collisions, and the buckets mean nothing to us

Pair feature vector: compressed version (11 values)

- 9 absolute differences $|z_i^{(k)} - z_{i+1}^{(k)}|$
- Trigram cosine (kept readable)
- Overall distance d_i

Models overview

Model	Setup	Interpretability
Decision tree	Gini, depth 5, min 20 to split, 5 per leaf, class weighted, threshold 0.5	Readable rules based on feature decisions
Logistic regression	Batch gradient descent, 10,000 iterations, lr 0.6, class weighted	Readable weights, one per feature
Neural network	11–128–64–32–16–8–1, ReLU, mini batch SGD (lr 0.001, batch 64), L2 0.01, dropout 0.2, early stopping	Black box, hard to interpret

- Class weights from inverse frequency (same author 0.583, switch 3.505)
- Thresholds tuned on the validation half to maximise F1 (logistic 0.68, NN 0.66)
- The validation half is used for tuning, the second half is used for final testing

Results: overall held out test (259,653 pairs)

Model	Acc.	Prec.	Rec.	F1	Bal. Acc.
Decision tree	0.850	0.462	**0.410**	0.435	0.666
Logistic regression	0.869	0.546	0.392	**0.456**	**0.669**
Neural network	**0.871**	**0.562**	0.368	0.445	0.661

- The three models are nearly tied (the neural network is not better than the linear model)
- The majority baseline already scores **0.859** accuracy, so accuracy alone is barely informative

Results depend almost entirely on difficulty

F1 / balanced accuracy on the held out test split (full training run)

Model	Easy	Medium	Hard
Decision tree	0.872 / 0.897	0.067 / 0.503	0.057 / 0.502
Logistic regression	0.869 / 0.890	0.078 / 0.515	0.040 / 0.503
Neural network	0.848 / 0.873	0.058 / 0.503	0.027 / 0.499

- **Easy:** strong, but author changes coincide with topic changes
- **Medium/hard:** balanced accuracy ≈ 0.50, i.e. chance, and below the trivial always switch F1

Always switch predicts "switch" for every pair: F1 $= 2r/(1+r)$, where r is the switch rate in each test level (easy 29.7%, medium 4.5%, hard 21.2%).

Interpretation: what the models actually used

Decision tree

- Root: character length difference (idx 1)
- Then average word length difference (idx 6)
- Trigram cosine appears at nearly every level
- Only 5 of 11 features used at all

Logistic regression weights

- Trigram cosine: -4.71 (more similar $\Rightarrow$ fewer switches)
- Character length difference: $+0.99$
- Overall distance: $+0.72$
- Frequent word and punctuation differences ≈ 0

The nine scalars are mostly surface statistics.

Handling class imbalance

Only 14.3% of pairs are switches, so "always same author" looks accurate.

1. Class weights
   Same author **0.583**
   Switch **3.505**
2. Tuned threshold
   Chosen on validation to maximise F1
3. Fair metrics
   Balanced accuracy, F1 and majority baseline

- No resampling: all 2.4M pairs are kept

What I am still working on (Future Work)

- **Feature ablation and importance**
- **Restoring n-gram compressed information**
- **Topic robust features:** Such as POS tagging usage, readability, vocabulary richness
- **Stronger models:** Such as random forest, gradient boosting

Summary

- Author Switch Detection system: 9 features + hashed tri-gram profile and distance: 11-value pair feature vector
- The results are baseline F1 (0.435 - 0.456)

Thank you. Questions?